

EIE3360 Integrated Project

Lecture 1 In-class questions

Q1: Predicting Titanic Survival

Objective:

Students will gather the Titanic dataset, preprocess the data, train a classification model to predict whether a passenger survived or not, and evaluate its performance.

1. Data Gathering

Dataset:

- Use the Titanic dataset, which is available from Kaggle.

Download the Dataset:

- You can download the dataset from Kaggle (or Blackboard) and load it into a pandas DataFrame.

```
import pandas as pd

# Load the Titanic dataset
df = pd.read_csv('titanic.csv')

# Display the first few rows of the dataset
print(df.head())
```

2. Data Preprocessing

Examine the dataset. Refer to Kaggle for the descriptions of the column.

How will you preprocess the data (which columns should be excluded, how to handle missing and zero values, etc.)?

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Handle missing values
df.fillna({'age':df['age'].median(), inplace=True)
df.fillna({'embarked':df['embarked'].mode()[0]}, inplace = True)
df.fillna({'fare':df['fare'].median(), inplace=True)
```

```
# Drop columns that won't be used
df.drop(['cabin', 'ticket', 'name', 'home.dest', 'boat', 'body'], axis=1, inplace=True)

# Convert categorical features to numerical values
label_encoder = LabelEncoder()
df['sex'] = label_encoder.fit_transform(df['sex'])
df['embarked'] = label_encoder.fit_transform(df['embarked'])

# Create new features
df['familySize'] = df['sibsp'] + df['parch'] + 1

# Split data into features and target
X = df.drop('survived', axis=1)
y = df['survived']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

3. Model Training

Choose a Classification Model:

- Use a classification model such as Logistic Regression, Decision Tree, Random Forest, or Gradient Boosting.

Train the Models:

- Train the four models using the training data in a similar way.

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

# Choose a RandomForestClassifier
modelRF = RandomForestClassifier(n_estimators=100, random_state=42)

# Train the model
modelRF.fit(X_train, y_train)
```

4. Model Evaluation

Evaluate the Model:

- Evaluate the four models on the test data using metrics such as accuracy, precision, recall, F1-score, and confusion matrix.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# Predict on the test set
y_pred = modelRF.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print(f'Accuracy: {accuracy}')
print(f'Precision: {precision}')
print(f'Recall: {recall}')
print(f'F1-Score: {f1}')
print(f'Confusion Matrix:\n{conf_matrix}')
```

Show your evaluation results of Logistic Regression, Decision Tree Classifier, Random Forest Classifier, and Gradient Boosting Classifier to TAs/Technicians.

Upload your .ipynb file to Blackboard.